The very first GPUs offered what is called a fixed-function pipeline, where the graphics hardware implemented a set of common graphics functions for performing transformations and applying graphical effects. This was the model used till DirectX 7, and OpenGL 1.5 and was the only system supported by the earliest NVIDIA cards.

In the early 2000s, GPU began opening up, allowing more and more of their functionality to be programmed by software developers using small bits of code called shaders rather than using a fixed set of functions in the graphics hardware.

OpenGL 2.0 and DirectX 8.0 facilitated the use of this new found flexibility in GPU hardware by including their own shader languages, GLSL (OpenGL Shading Language) for OpenGL and HLSL (High Level Shader Language) for DirectX. Code could be written in these languages and then deployed to be run in the GPU.

GPUs continued opening up to the point that modern GPUs are in essence highly-parallel CPUs. Today you have frameworks such as CUDA and OpenCL that allow a developer to utilise the power of a GPU to perform tasks that would traditionally be performed by a CPU.

To give you an idea, currently the latest NVIDIA GPU architecture is Maxwell, which powers its current top of the line GTX 980 GPU. This GPU has 16 streaming multiprocessors, each of which has 4 warp schedulers, which in turn have 32 CUDA cores. Leave the confusing technology aside, and think of this as equivalent to a 16-core CPU, where each CPU has the equivalent of Intel's HyperThreading, but instead of making a single core appear as 2, it allows for a single core to appear as 128!

In the meantime we have seen the release of OpenGL 3.x and 4.x, and the release of DirectX 9.x, 10.x and 11.x. While graphics APIs continued to evolve, they also abstracted away the true power of modern GPUs in order to provide an API that was designed for old hardware. They also add an overhead, in terms of CPU usage that limits what is possible through the use of these APIs.

The Growing Consensus

The uniformity and simplicity of old OpenGL and DirectX comes with an overhead that can sometimes be limiting.

This is more true perhaps of mobile platforms such as Apple's iOS, where the CPU and GPU are both limited, and every bit of inefficiency means lower battery life, or less optimum use of the available hardware. Imagine a powerful GPU that is capable of brilliant graphical fidelity, but is let down by the slower CPU of a mobile device.

The developer is using the GPU through a layer of software (OpenGL 4.x or DirectX 11.x) that need to process each command sent to the GPU using those APIs. However these commands themselves use up the CPU to the point where we just can't send instructions to the GPU fast enough to make full use of it.

It is for this reason that Apple created their Metal API for iOS, giving game developers for iOS devices more direct control over the graphics hardware, and the capability for better performance, or better visuals with the same hardware.

In fact console developers creating games for the Sony PlayStation and the Microsoft XBox have had access to low-level APIs because of which you can see such hardware achieve levels of graphical fidelity that are just not possible with desktop computers with much better specifications.

Both Mantle, by AMD, and DirectX 12 by Microsoft, which preceded Apple's Metal are built around the same principles. Think of them as a reboot of the graphics APIs which were designed for another time period and are now showing their age.

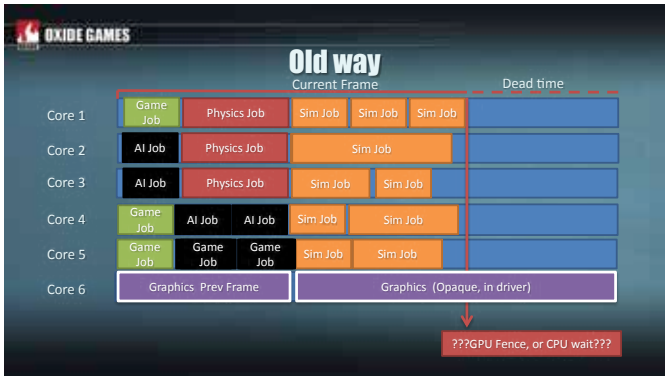
This was also a little unfortunate, since Mantle (when announced) was an AMD-only technology and would only work on AMD hardware, that too only on Windows. DirectX 12 is a Microsoft-only technology that would only work with Microsoft platforms such as Windows, Windows Phone and the XBox. Metal was an Apple-only API that would only run on Apple platforms, and currently is only available for iOS.

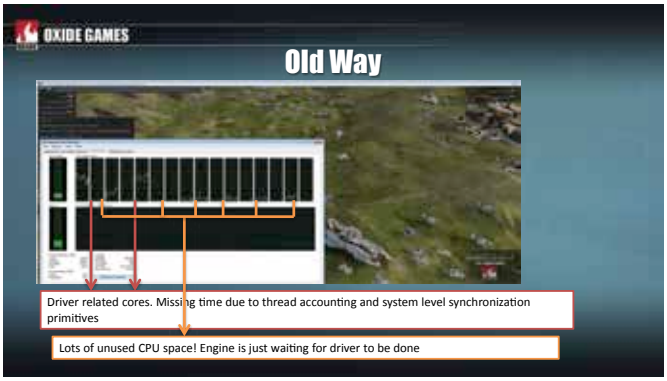
What about a standard graphics API that could work on all operating systems, whether it is OSX, Linux, Windows, iOS or Android, and on all kinds of hardware whether by AMD, NVIDIA, or Intel. OpenGL had always been the solution that could go anywhere due to its open nature, but it hasn't suddenly disappeared in a puff of smoke now that these newer APIs are available. It will continue to be possible to use OpenGL to create games and other applications needing hardware acceleration.

The Future of OpenGL

The obvious solution then is to simply create a new version of OpenGL with the same design goals as these newer closer-to-the-metal APIs, while remaining vendor and platform-neutral as it is now.

Many developers working with OpenGL have talked about the many issues faced by them in OpenGL, and the need for a clean break, an entirely new graphics API developed using the same open approach as OpenGL, but built for modern graphics chipset architectures, and with modern graphics development in mind.





Vulkan is better at spreading load across multiple cores